ELDERLY CARE MONITORING SYSTEM WITH EMERGENCY ALERTS

Git & Docker Technical Assignment Report

Submitted in the partial fulfilment for the Course of

CSA1012 SOFTWARE ENGINEERING

to the award of the degree of

BACHELOR OF ENGINEERING

IN

INFORMATION TECHNOLOGY

Submitted by

MAHIMA (192521280)

Under the Supervision of

Dr. K. Anita Davamani

SIMATS ENGINEERING

SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

November 2025

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

Engineer to Excel

DECLARATION

I, **MAHIMA (192521280),** of the **INFORMATION TECHNOLOGY,** Saveetha Institute of Medical and Technical Sciences, Saveetha University, Chennai, hereby declare that the Capstone Project Work entitled **'ELDERLY CARE MONITORING SYSTEM WITH EMERGENCY ALERTS** 'is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with principles of engineering ethics.

Place: Chennai

Date: 28-11-2025

Signature of the Students with Names

MAHIMA (192521280)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

Engineer to Excel

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled "**ELDERLY CARE MONITORING SYSTEM WITH EMERGENCY ALERTS**" has been carried out by **Mahima (192521280)** under the supervision of **Dr. Rameswari** and is submitted in partial fulfilment of the requirements for the current semester of the B. Tech **INFORMATION TECHNOLOGY** program at Saveetha Institute of Medical and Technical Sciences, Chennai.

SIGNATURE

Program Director
Department of IT
Saveetha School of Engineering
SIMATS

SIGNATURE

Professor
Department of IT
Saveetha School of Engineering
SIMATS

Submitted for the Project work Viva-Voce held on _____
.

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude to all those who supported and guided us throughout the successful completion of our Capstone Project. We are deeply thankful to our respected Founder and Chancellor, Dr. **N.M. Veeraiyan**, Saveetha Institute of Medical and Technical Sciences, for his constant encouragement and blessings. We also express our sincere thanks to our Pro-Chancellor, **Dr. Deepak Nallaswamy Veeraiyan**, and our Vice-Chancellor, **Dr. S. Suresh Kumar**, for their visionary leadership and moral support during this project.

We are truly grateful to our Director, **Dr. Ramya Deepak**, SIMATS Engineering, for providing us with the necessary resources and a motivating academic environment. Our special thanks to our Principal, **Dr. B. Ramesh** for granting us access to the institute's facilities and encouraging us throughout the process. We sincerely thank our Head of the Department, **Dr K. Malathi** for her continuous support, valuable guidance, and constant motivation.

We are especially indebted to our guides, **Dr. K. Anita Davamani** or their creative suggestions, consistent feedback, and unwavering support during each stage of the project. We also express our gratitude to the Project Coordinators, Review Panel Members (Internal and External), and the entire faculty team for their constructive feedback and valuable inputs that helped improve the quality of our work. Finally, we thank all faculty members, lab technicians, our parents, and friends for their continuous encouragement and support.

Signature With Student Name

MAHIMA (192521280)

INDEX

1. Problem Analysis

The proposed system, Elderly Care Monitoring System with Emergency Alerts, is intended to provide a software-based monitoring layer for elderly users who may require timely assistance during emergencies. The central problem is that an elderly person may experience an unsafe event such as a fall or another emergency while a caregiver is not immediately present. A monitoring application can collect status information, expose health/status endpoints, maintain emergency-alert information, and provide a foundation for future sensor integration.

The implementation used for this technical assignment is deliberately lightweight so that the version-control and containerization workflow can be demonstrated clearly. A Flask-based REST backend exposes the system status, health status, and emergency-alert endpoints. The current alert model includes a fall-detection example and supports creation of a new emergency alert through a POST request.

1.1 Objectives

- Provide a simple backend for an elderly-care monitoring application.
- Expose a health endpoint that can be used to verify service availability.
- Provide an emergency-alert endpoint for retrieving active alerts.
- Support creation of new emergency alerts through a REST API.
- Maintain the source code using Git with a main/develop/feature workflow.
- Containerize the backend using Docker so the same application can run in an isolated environment.
- Provide reproducible deployment and testing evidence suitable for academic evaluation.

1.2 Functional Requirements

ID	Requirement	Expected Behaviour
FR-01	System status	The root endpoint reports that the Elderly Care Monitoring System is running.
FR-02	Health monitoring	The /health endpoint returns a healthy service status.
FR-03	Alert retrieval	The /alerts endpoint returns the current emergency-alert collection.

FR-04	Alert creation	A POST request to /alerts creates and returns a new active emergency alert.
FR-05	Version control	Project changes are tracked through Git branches and meaningful commits.
FR-06	Container execution	The Flask backend runs successfully inside a Docker container on port 5000.

1.3 Expected Outcomes

The expected outcome is a reproducible project in which the source code can be versioned through Git and deployed through Docker. The implementation demonstrates that the application can be started locally, tested through HTTP endpoints, built into a Docker image, and executed as a container. The architecture also provides a base for later integration of real sensors, persistent storage, authentication, caregiver notifications, and monitoring dashboards.

2. Project Structure Design

The repository is organized to keep application code, database initialization, container configuration, and documentation separate. This separation improves maintainability because changes to application logic do not require reorganizing deployment configuration.

Repository structure:

```
elderly-care-monitoring/
├── backend/
│   ├── app.py
│   ├── requirements.txt
│   └── README.md
├── database/
│   └── init.sql
├── Dockerfile
├── docker-compose.yml
├── .dockerignore
├── .gitignore
└── README.md
```

2.1 Folder and File Justification

Component	Purpose	Justification
backend/	Application source code and Python dependencies.	Keeps runtime logic independent from infrastructure files.
backend/app.py	Flask application and REST endpoints.	Contains the executable service logic.
backend/requirements.txt	Python package dependencies.	Allows deterministic dependency installation during Docker builds.
backend/README.md	Backend-specific documentation.	Explains the backend component separately.
database/	Database-related initialization.	Provides a clear place for persistent-data setup as the system evolves.
database/init.sql	Initial SQL/schema definition.	Supports future database-backed monitoring and alert persistence.
Dockerfile	Application image definition.	Defines a reproducible container build.
docker-compose.yml	Multi-service orchestration definition.	Provides a single command-based deployment model.
.gitignore	Files excluded from Git tracking.	Prevents generated/cache/local files from entering version control.
.dockerignore	Files excluded from Docker build context.	Reduces unnecessary build context and image-build overhead.
README.md	Project-level documentation.	Provides setup, usage, and workflow information.

3. Git Repository Initialization

Git was initialized at the project root so that all source, configuration, documentation, and deployment files could be versioned together. The repository was initialized using `git init`, after which the default branch was renamed to `main` to establish a stable primary branch.

3.1 Initialization Procedure

1. Navigate to the project root directory.
2. Run `git init` to create the local .git repository.
3. Run `git branch -M main` to use main as the stable branch.
4. Run `git status` to inspect untracked files.
5. Stage project files with `git add .`.
6. Create the first meaningful commit using `git commit -m "Initial project structure and backend setup"`.
7. Verify the history using `git log --oneline`.

3.2 Essential Git Files

File / Directory	Role
.git/	Hidden Git metadata directory containing repository history, references, and configuration.
.gitignore	Specifies files that should not be tracked, such as Python caches, virtual environments, and local secrets.
README.md	Human-readable project documentation.
.dockerignore	Controls which files are excluded from Docker build context.

3.3 Evidence

The recorded terminal evidence shows successful repository initialization, a clean transition to the main branch, staging of nine project files, and creation of the root commit `fd31f19` with the message `Initial project structure and backend setup`.

```
C:\Users\welcome\OneDrive\Desktop\elderly-care-monitoring>git add .

C:\Users\welcome\OneDrive\Desktop\elderly-care-monitoring>git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   .dockerignore
        new file:   .gitignore
        new file:   Dockerfile
        new file:   README.md
        new file:   backend/README.md
        new file:   backend/app.py
        new file:   backend/requirements.txt
        new file:   database/init.sql
        new file:   docker-compose.yml


C:\Users\welcome\OneDrive\Desktop\elderly-care-monitoring>
```

Figure 1. Git repository initialization and first commit

4. Git Branching Strategy

A simplified Git Flow-style model was selected because the project has a stable branch, an integration branch, and feature-specific development. The strategy avoids direct development on main and allows individual functionality to be developed and reviewed before integration.

main → develop → feature/*

4.1 Branch Roles

Branch	Role	Rule
main	Stable submission/release branch.	Only tested and accepted changes should reach main.
develop	Integration branch.	Feature branches are merged here after development.
feature/emergency-alerts	Emergency-alert implementation.	Contains changes related specifically to emergency alert creation.

4.2 Why This Strategy Fits the Project

- Emergency alert functionality can be isolated from the stable codebase.
- Multiple future features such as authentication, caregiver notifications, sensor ingestion, and database persistence can use independent feature branches.
- The develop branch provides an integration point before a stable release.
- The strategy supports collaborative work and reduces accidental changes to the stable branch.
- The model is simple enough for a student project while still demonstrating professional branching concepts.

5. Version Control Workflow

The Git workflow was demonstrated through meaningful repository operations rather than creating empty branches. The emergency-alert feature was developed on a dedicated feature branch, committed, and then merged into develop. The repository was subsequently synchronized with GitHub.

5.1 Operations Performed

Operation	Command	Purpose	Observed Result
Initialize	git init	Create local repository.	Repository initialized successfully.
Rename branch	git branch -M main	Create stable primary branch name.	main established.
Stage	git add .	Prepare files for commit.	Nine project files staged.
Commit	git commit -m ...	Create traceable project checkpoint.	Root commit fd31f19 created.
Create develop	git branch develop	Create integration branch.	develop created.
Switch	git switch develop	Move to integration branch.	develop active.
Feature branch	git switch -c feature/emergency-alerts	Isolate emergency-alert development.	Feature branch created.

Feature commit	git commit -m "Add emergency alert creation endpoint"	Record completed feature.	Commit b3e235d created.
Merge	git merge feature/emergency-alerts	Integrate feature into develop.	Fast-forward merge completed.
Synchronize	git push -u origin main/develop	Publish repository to GitHub.	Local branches synchronized.

5.2 Conflict Resolution

No merge conflict occurred during the demonstrated workflow. The emergency-alert feature could be integrated with a fast-forward merge because develop had not diverged from the feature branch. Therefore, no artificial conflict was introduced solely for demonstration. In a larger team, Git would identify conflicting edits and require manual resolution followed by testing and a conflict-resolution commit.

5.3 Workflow Summary

Working tree → feature branch → meaningful commit → develop merge → GitHub synchronization

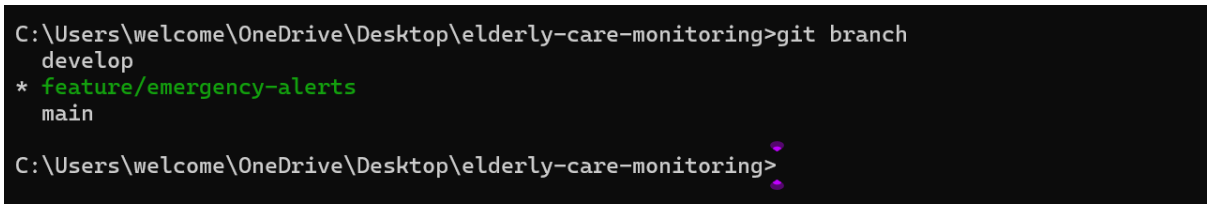

Figure 2. Git workflow used for the emergency-alert feature

6. Commit History Analysis

The commit history contains meaningful checkpoints that describe the purpose of each change. The two principal commits captured during implementation are:

Commit	Message	Meaning
fd31f19	Initial project structure and backend setup	Establishes the repository, application structure, Docker configuration files, and initial backend.

b3e235d	Add emergency alert creation endpoint	Adds POST-based emergency alert creation to the Flask backend.

The messages use imperative, action-oriented wording and identify the scope of the change. This improves maintenance because a future developer can understand the reason for a commit without opening every changed file. The graph also demonstrates that the emergency-alert commit became part of develop while the original main commit remained the stable base.

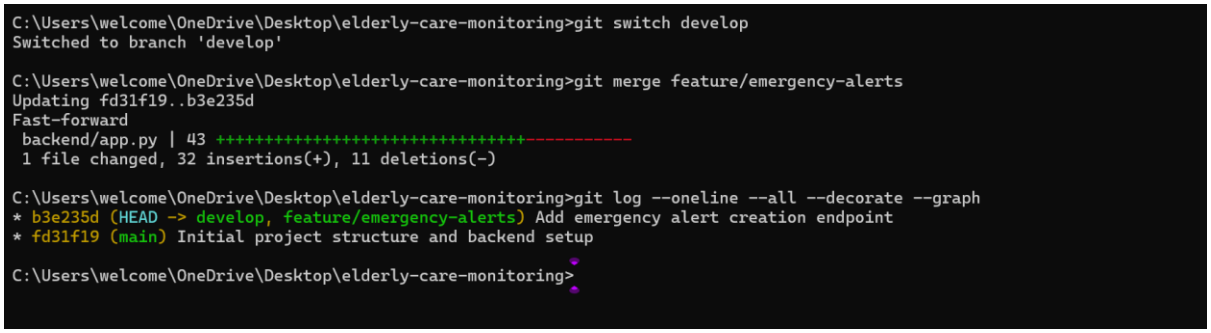

Figure 3 git log --oneline --all --decorate --graph` showing branch relationships.

8. Dockerfile Development

The Dockerfile uses a lightweight Python base image and separates dependency installation from source-code copying. This makes the image definition straightforward and allows Docker's build cache to reuse dependency layers when only application source files change.

```
FROM python:3.12-slim

WORKDIR /app

COPY backend/requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY backend/ .

EXPOSE 5000

CMD ["python", "app.py"]
```

8.1 Instruction Analysis

Instruction	Purpose
FROM python:3.12-slim	Uses a smaller Python 3.12 Linux base image.
WORKDIR /app	Sets the container working directory.
COPY backend/requirements.txt .	Copies dependency definitions before source code.
RUN pip install --no-cache-dir -r requirements.txt	Installs Python dependencies without retaining pip cache.
COPY backend/ .	Copies Flask application source into the image.
EXPOSE 5000	Documents the port used by the Flask application.
CMD ["python", "app.py"]	Starts the application when the container runs.

8.2 Build Evidence

The Docker image was successfully built using `docker build -t elderly-care-monitoring .`. The build completed with all build stages finished and the resulting image was named `elderly-care-monitoring`.

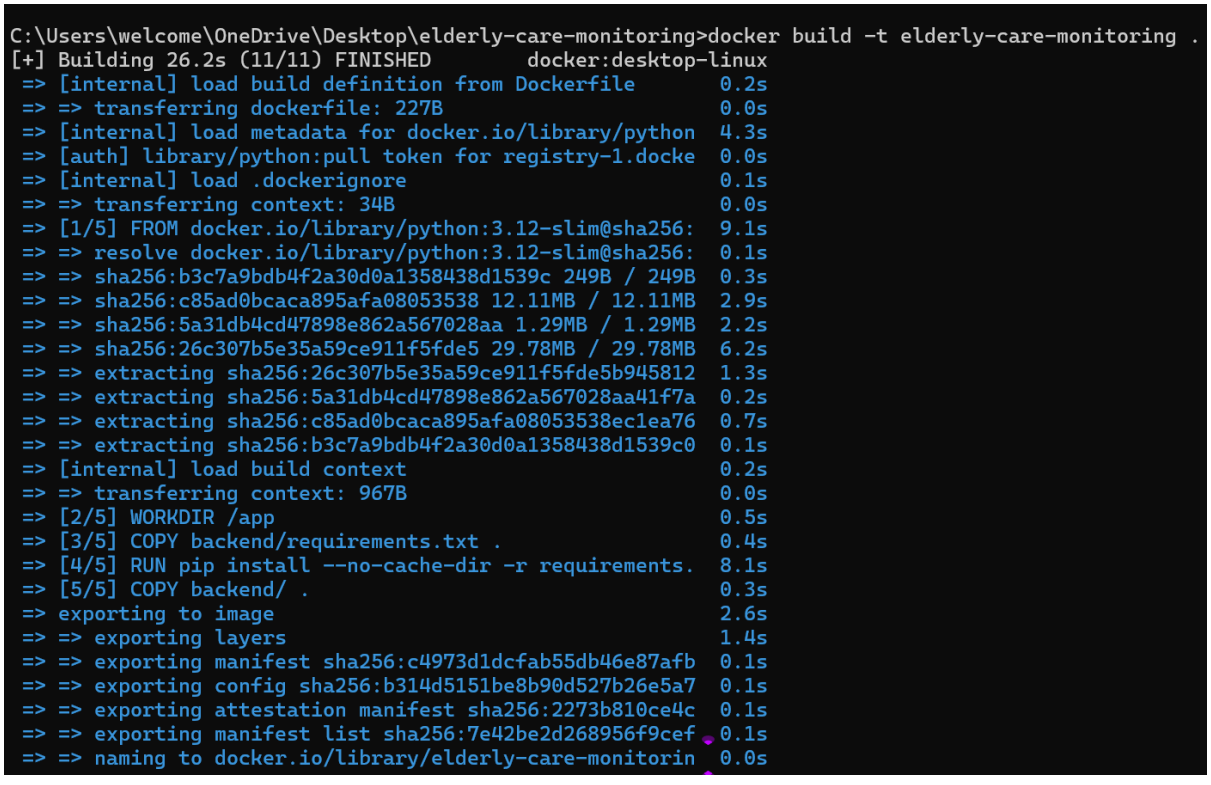

Figure 4. Successful Docker image build

9. Docker Compose Design

Docker Compose is used to describe the application as a set of services rather than requiring each container to be started manually. For the current assignment, the Flask service is the core executable component. A database service can be included as the persistence layer for alerts and elderly monitoring data.

Recommended final Compose configuration for the submission:

```
services:
  app:
    build: .
    container_name: elderly-care-app
    ports:
      - "5000:5000"
    environment:
      APP_ENV: development
    depends_on:
      - db


 db:
    image: postgres:16
    container_name: elderly-care-db
    environment:
      POSTGRES_DB: elderlycare
      POSTGRES_USER: elderly
      POSTGRES_PASSWORD: elderly_password
    volumes:
      - elderlycare_data:/var/lib/postgresql/data


volumes:
  elderlycare_data:
```

10. Container Deployment and Testing

The application was successfully deployed as a Docker container. The image was built and then executed with port 5000 mapped from the host to the container. The running-container evidence showed the container status as Up and the mapping `0.0.0.0:5000->5000/tcp`.

10.1 Deployment Commands

8. `docker build -t elderly-care-monitoring .` — builds the application image.
9. `docker run -d -p 5000:5000 --name elderly-care-app elderly-care-monitoring` — starts the application container.

10. `docker ps` — verifies that the container is running and exposes port 5000.

11. `docker logs elderly-care-app` — verifies application activity and HTTP requests.

10.2 Test Cases

Test ID	Request	Expected Result	Observed
TC-01	GET /	System status reports running.	Successful before and during containerized testing.
TC-02	GET /health	Health response reports healthy.	Successful during local API testing.
TC-03	GET /alerts	Emergency alert collection is returned.	Successful; Docker logs recorded HTTP 200.
TC-04	POST /alerts	New active emergency alert is created.	Endpoint implemented and committed; include API-client evidence if available.
TC-05	docker ps	Container status is Up with port mapping.	Successful; container was Up and port 5000 was mapped.

10.3 Log Interpretation

The container log included a successful `GET /alerts HTTP/1.1` request with HTTP status 200. The log also contained unrelated browser-generated requests returning 404. Those requests are not application failures because they targeted routes that are outside the project's API. The relevant application endpoint returned 200, demonstrating successful request handling inside the container.

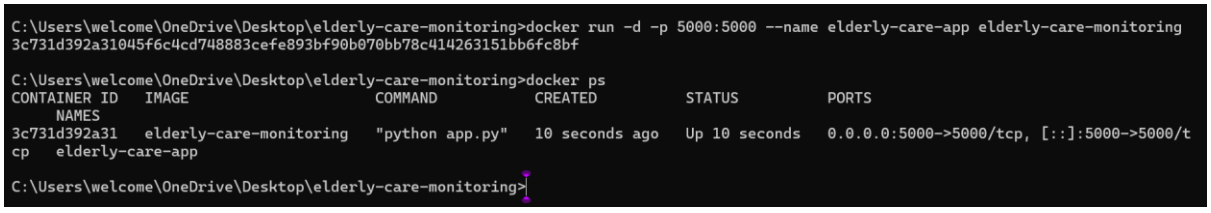

Figure 5. Running Docker container using `docker ps`.

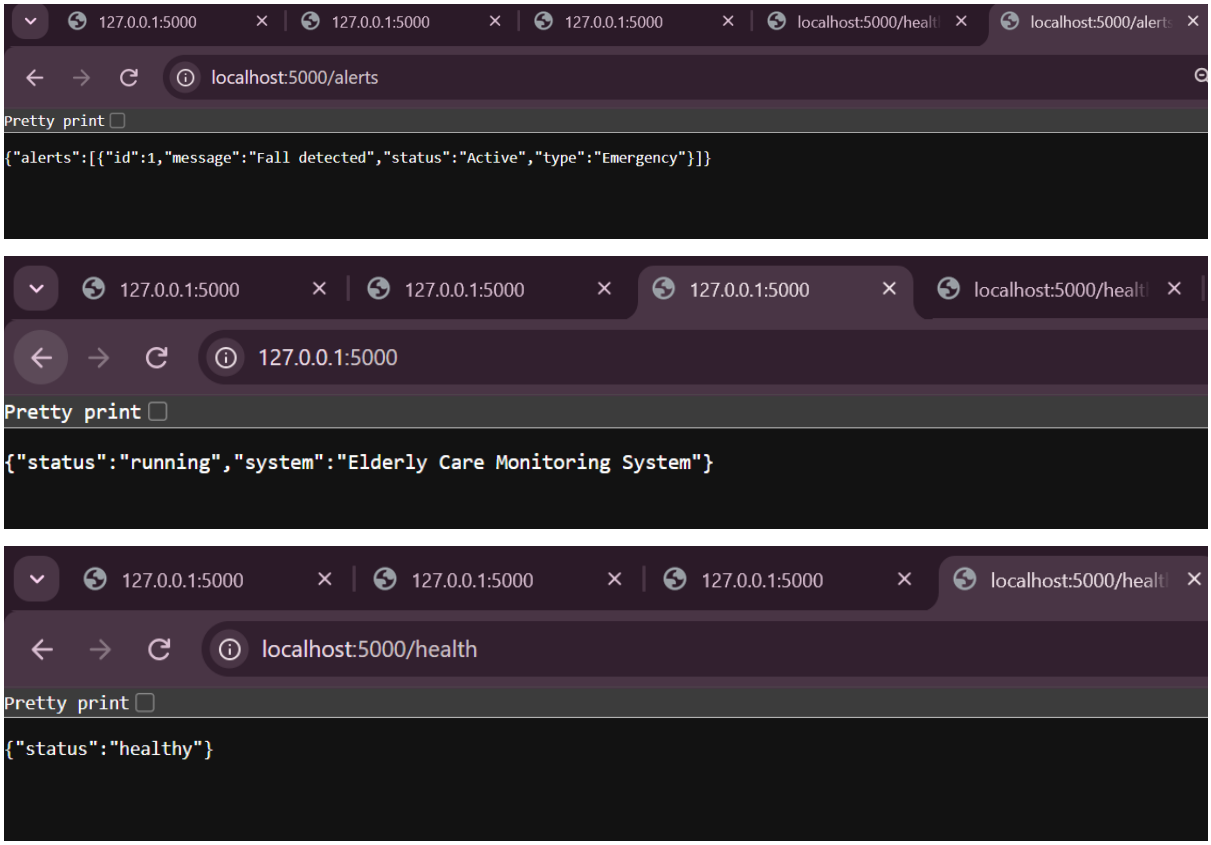

Figure 6. Container logs showing successful `/alerts`, `/health and excution of app request with HTTP 200.

11. Conclusion

The Elderly Care Monitoring System with Emergency Alerts demonstrates how software-engineering practices can be applied to a safety-oriented monitoring application. The project was organized as a maintainable repository, versioned using Git, developed through a main/develop/feature branching model, and synchronized with GitHub. The emergency-alert functionality was implemented as a meaningful feature and integrated into the develop branch.

Docker was then used to package the Flask backend together with its Python runtime and dependencies. The Docker image was built successfully and executed as a container with port 5000 exposed to the host. Testing through the HTTP API and container logs demonstrated successful request handling, including an HTTP 200 response for the emergency-alert endpoint.

The combination of Git and Docker provides a strong foundation for future development. Git gives the project traceability, controlled collaboration, and recoverable history, while Docker provides reproducible execution and deployment. With the addition of Compose verification, persistent database integration, authentication, caregiver notifications, automated tests, and CI/CD, the prototype can evolve into a more complete elderly-care platform.

15. References

[1] Git Documentation. Git version control documentation and command reference.

[2]Docker Documentation. Docker Engine, Dockerfile and Docker Compose documentation.

[3] Turnbull, J., The Docker Book: Containerization is the New Virtualization.

[4] Bass, L., Clements, P., and Kazman, R., Software Architecture in Practice.

[5] Humble, J. and Farley, D., Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation.